

SPRINT 4 - BigQuery Avanzado y Analytics Engineering

Nivel 1 - Optimización de almacenamiento y rendimiento

Caso 1. Diagnóstico de una consulta de alto coste

Contexto

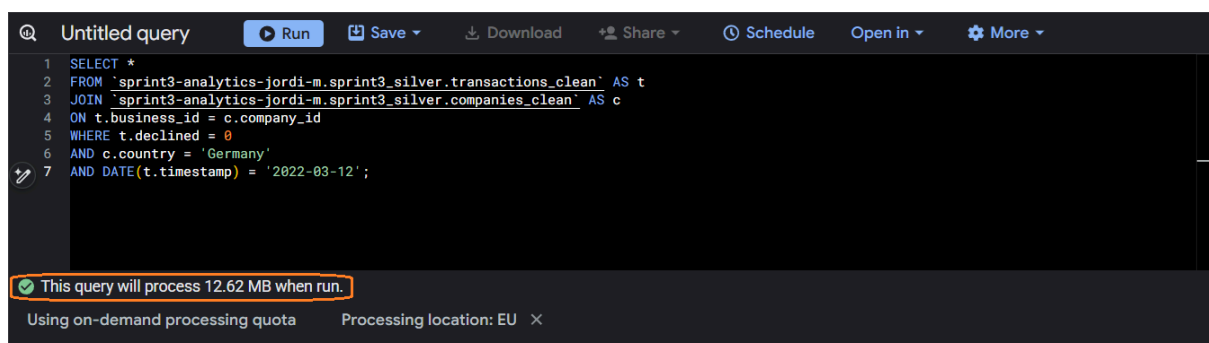
El responsable regional de Alemania necesita consultar con urgencia las transacciones realizadas el 12 de marzo de 2022.

Los informes actuales combinan información de transacciones y empresas mediante operaciones JOIN. Sin embargo, existen sospechas de que el sistema está realizando lecturas completas de tablas (Full Table Scan), generando costes elevados y tiempos de respuesta innecesarios.

Objetivo

Analizar el coste estimado de una consulta que combina transacciones y empresas filtrando por:

- País: Alemania
- Fecha: 12/03/2022
- Ventas: Declined = 0



Esta consulta, al leer toda la información de dos tablas, tiene un coste de proceso bastante alto (12.62 MB aproximadamente).

Caso 2. Re-arquitectura del modelo físico mediante particionado y clustering

Contexto

Tras el análisis inicial se confirma que los filtros por fecha y empresa siguen provocando lecturas masivas de datos.

Para reducir costes y mejorar el rendimiento se decide rediseñar físicamente la tabla de hechos principal.

Objetivo

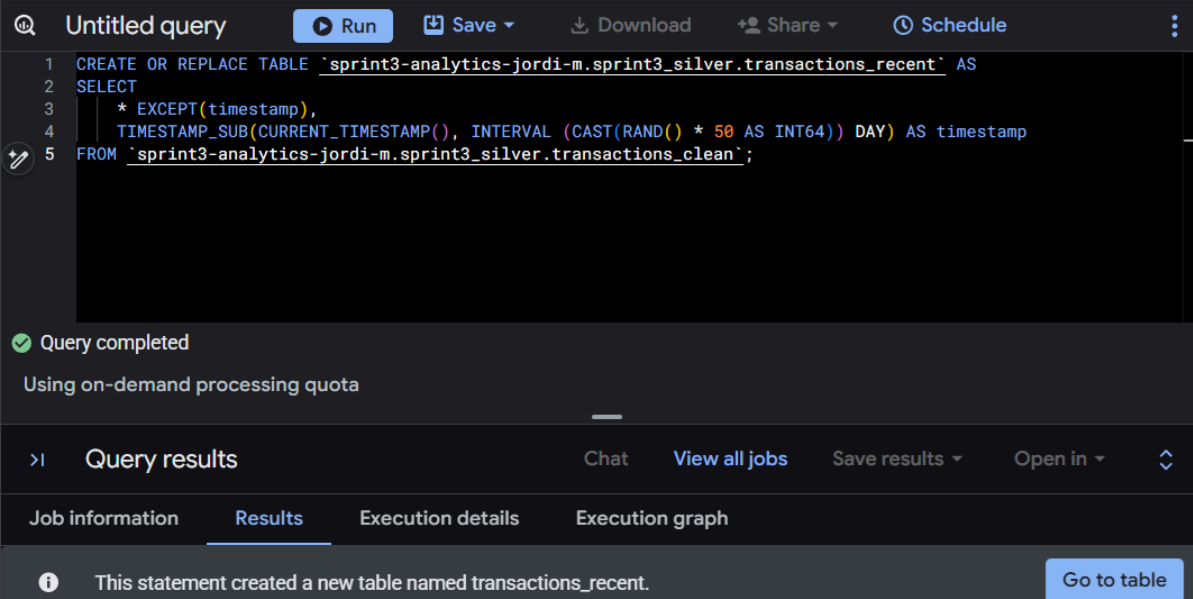
Crear una tabla optimizada para consultas analíticas frecuentes mediante:

- Particionado por fecha.
- Clustering por empresa.

Solución aplicada

Generación de datos recientes

Debido a las limitaciones del entorno sandbox de BigQuery, se generó una versión de los datos con fechas dentro de los últimos 50 días.



```
1 CREATE OR REPLACE TABLE `sprint3-analytics-jordi-m.sprint3_silver.transactions_recent` AS
2 SELECT
3   * EXCEPT(timestamp),
4   TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL (CAST(RAND() * 50 AS INT64)) DAY) AS timestamp
5 FROM `sprint3-analytics-jordi-m.sprint3_silver.transactions_clean`;
```

✓ Query completed

Using on-demand processing quota

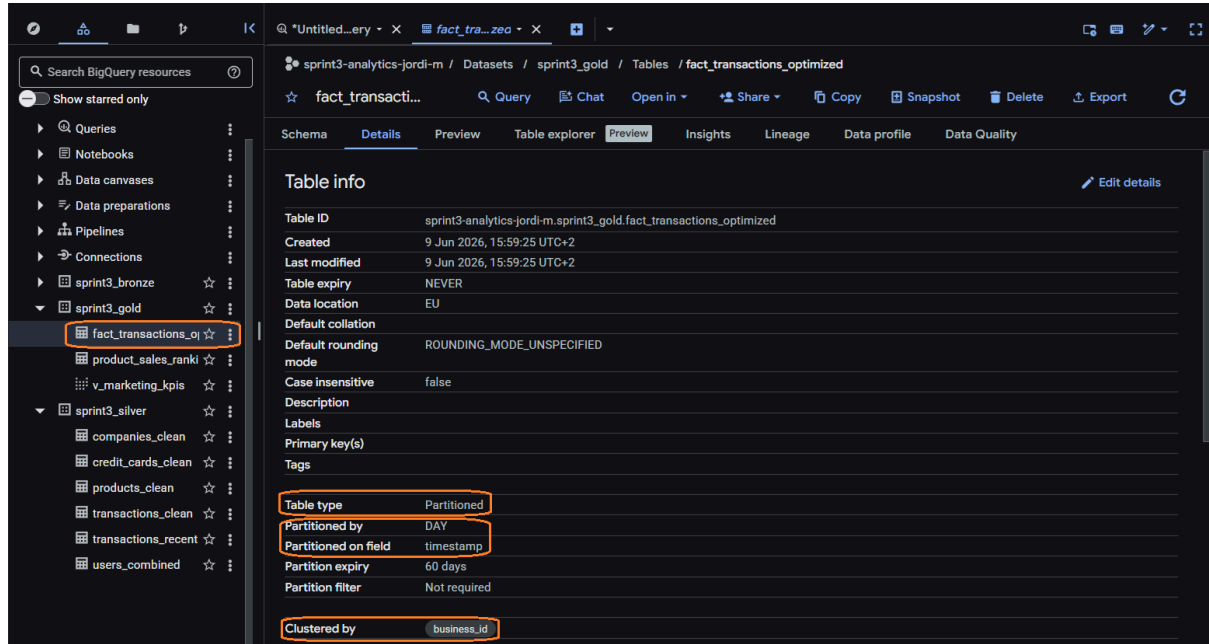
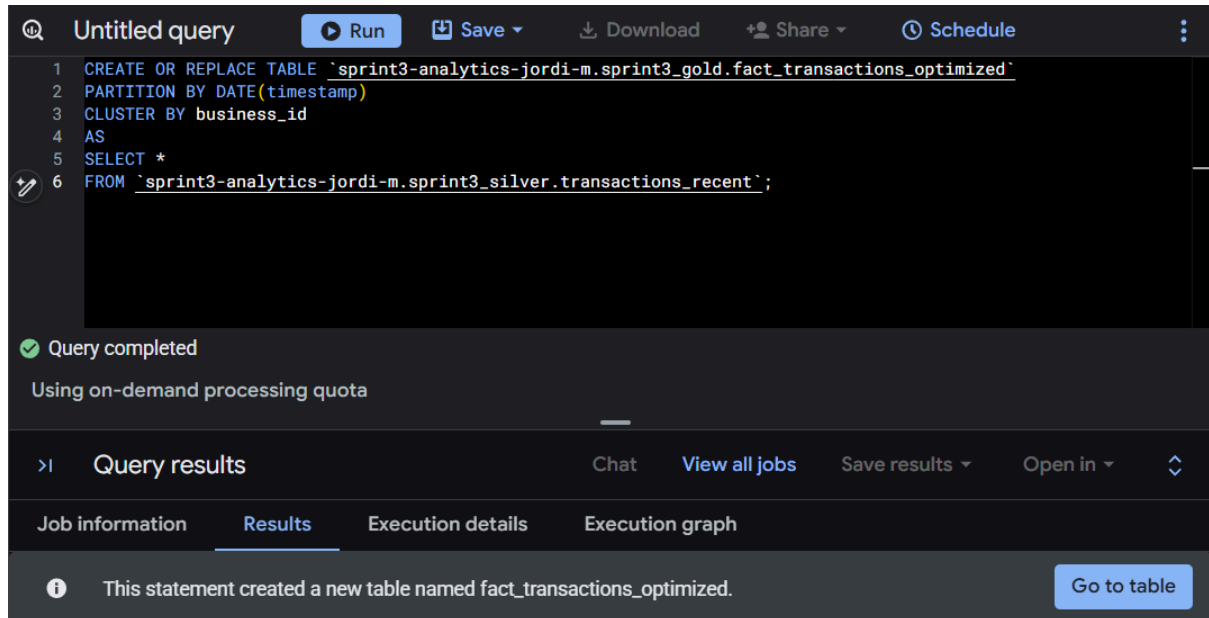
> Query results Chat View all jobs Save results Open in

Job information Results Execution details Execution graph

i This statement created a new table named transactions_recent. Go to table

Optimización física

Se creó una nueva tabla con particionado por fecha de transacción, y clustering por identificador de empresa.



Como podemos ver en la imagen, la nueva tabla particionada se ha creado satisfactoriamente, y en la parte inferior de la imagen podemos ver las características en base a qué columnas hemos particionado y agrupado. Esto nos permitirá hacer consultas menos costosas cuando usemos filtros.

Caso 3. Benchmark de rendimiento

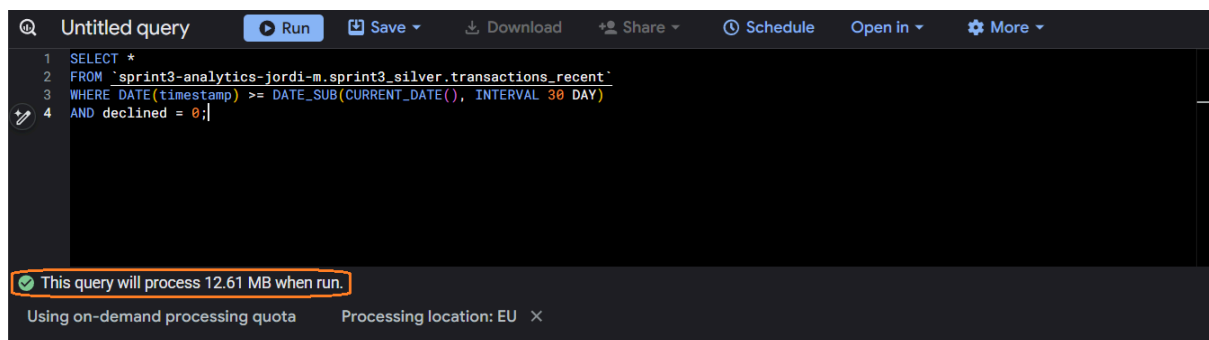
Contexto

Una vez creada la nueva arquitectura, es necesario cuantificar la mejora obtenida.

Objetivo

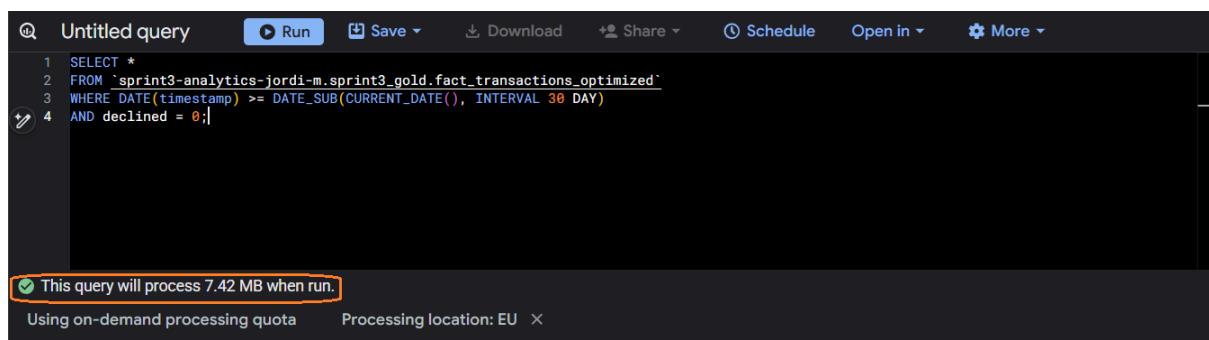
Comparar el coste estimado de la misma consulta sobre:

- Tabla no optimizada.



Vemos que si queremos ver toda la información de los últimos 30 días de la tabla 'sprint3_silver.transaction_recent', el coste de procesamiento será de aproximadamente 12.61 MB.

- Tabla optimizada.



La misma consulta aplicada a la tabla 'sprint3_gold.fact_transactions_optimized' tiene un coste de procesamiento de aproximadamente 7.67 MB. De esta manera, donde ahorramos aproximadamente un 40% en la misma consulta.

Calculamos el porcentaje de ahorro con la siguiente fórmula:

Porcentaje de ahorro = (Bytes_original - Bytes_optimizada) / Bytes_original * 100

Caso 4. Optimización de consultas recurrentes mediante Materialized Views

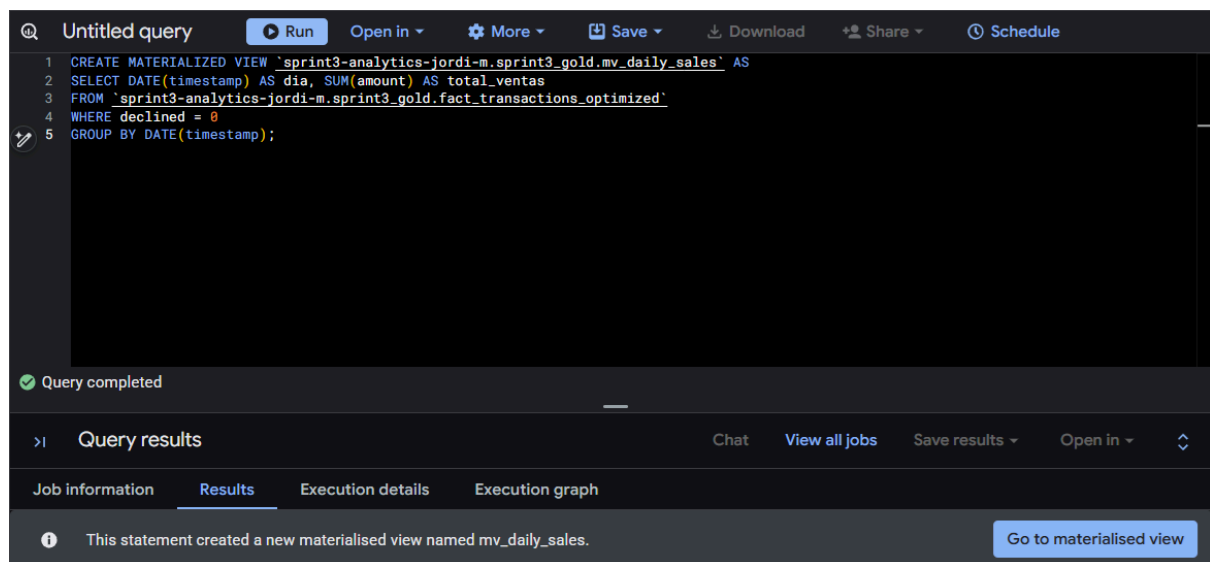
Contexto

Los cuadros de mando ejecutan constantemente el mismo cálculo: ventas diarias agregadas.

Recalcular millones de registros en cada actualización genera un consumo innecesario de recursos.

Objetivo

Implementar una estrategia de caché inteligente mediante Materialized Views.



Creamos una vista materializada sobre la tabla de hechos optimizada (particionada y clusterizada) de los ejercicios anteriores, combinando optimización física de almacenamiento con caché de agregaciones para minimizar tiempos de respuesta y costes de consulta.

The screenshot shows the Google Cloud BigQuery interface. On the left, a sidebar lists the project hierarchy: sprint3-analytics-jordi-m > sprint3_gold > mv_daily_sales. The 'mv_daily_sales' view is highlighted with a red box. The main panel displays the 'Materialised view info' for 'mv_daily_sales'. The view was created on 9 Jun 2026, 16:15:29 UTC+2, last modified on 9 Jun 2026, 16:15:33 UTC+2, and expires on 8 Aug 2026, 16:15:29 UTC+2. It has a refresh interval of 1800000 ms and is enabled. The view is defined as a materialized view over the 'mv_daily_sales' table.

Property	Value
Materialised view ID	sprint3-analytics-jordi-m.sprint3_gold.mv_daily_sales
Created	9 Jun 2026, 16:15:29 UTC+2
Last modified	9 Jun 2026, 16:15:33 UTC+2
Materialised view expiry	8 Aug 2026, 16:15:29 UTC+2
Last refresh time	9 Jun 2026, 16:15:29 UTC+2
Refresh enabled	true
Refresh interval (ms)	1800000
Max staleness	
Allow non-incremental definition	false
Replica(s)	
Description	
Labels	
Primary key(s)	
Tags	

Aquí podemos ver como la vista se ha creado satisfactoriamente y sus características.

The screenshot shows the Google Cloud BigQuery interface with a query executed. The query is: `SELECT * FROM `sprint3-analytics-jordi-m.sprint3_gold.mv_daily_sales`;`. The query completed successfully, using on-demand processing quota. The results are displayed in a table with 13 rows shown. The table has columns 'dia' and 'total_ventas'. The results show daily sales data for the period from 2026-05-29 to 2026-05-30. The total number of results is 51, as indicated by the '1 - 50 of 51' text at the bottom right.

Row	dia	total_ventas
1	2026-05-29	507239.29
2	2026-05-23	500249.64
3	2026-05-03	524929.14
4	2026-04-21	541072.76
5	2026-06-09	257318.66
6	2026-05-24	512126.51
7	2026-05-10	518431.92
8	2026-04-29	509488.32
9	2026-05-19	498873.24
10	2026-05-11	523027.12
11	2026-04-26	507849.84
12	2026-05-12	531349.81
13	2026-05-30	532705.98

Aquí podemos previsualizar una consulta de la vista completa. También vemos que tiene 51 registros.

Untitled query Run Open in More Save Download Share Schedule

```
1 SELECT *
2 FROM `sprint3-analytics-jordi-m.sprint3_gold.mv_daily_sales`;
```

Query completed
Using on-demand processing quota

Query results Chat View all jobs Save results Open in

Job information Results Visualisation JSON Execution details Execution graph

Job ID	sprint3-analytics-jordi-m:EU.bquxjob_14bcc5cf_19eacbf2490 (See performance details)
User	jj.montseny@gmail.com
Location	EU
Creation time	9 Jun 2026, 16:17:49 UTC+2
Start time	9 Jun 2026, 16:17:49 UTC+2
End time	9 Jun 2026, 16:17:50 UTC+2
Duration	0 sec
Bytes processed	1.2 KB
Bytes billed	10 MB
Slot milliseconds	45
Job priority	INTERACTIVE
Use legacy SQL	false
Destination table	Temporary table
Index usage mode	UNUSED
Index unused reasons	There is no index configuration for the base table `sprint3-analytics-jordi-m:sprint3_gold.fact_transactions_optimized`.

Esta vista materializada, al guardar los resultados de los cálculos en la memoria, sólo actualiza la consulta si entran datos nuevos, por lo que el coste computacional es mucho menor.

Nivel 2 - SQL Analítico Avanzado

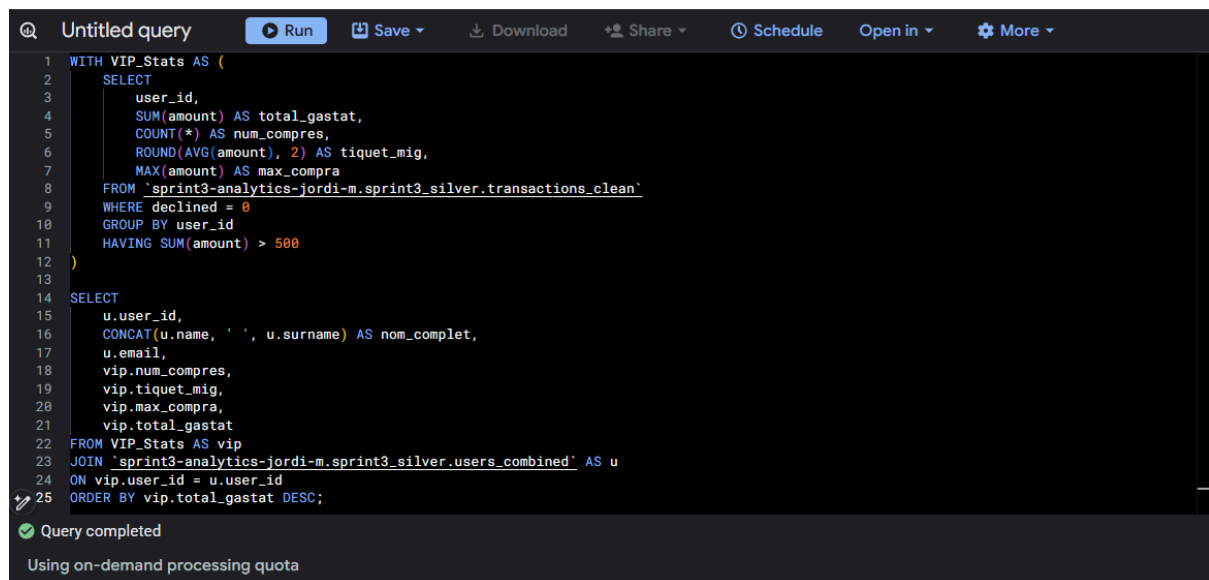
Caso 1. Identificación y perfilado de clientes VIP

Contexto

El equipo de Marketing necesita identificar a los clientes de mayor valor para futuras campañas de fidelización.

Objetivo

Detectar usuarios con más de 500 € de gasto acumulado y analizar su comportamiento.

A screenshot of a SQL query editor interface. The title bar says "Untitled query" and includes buttons for "Run", "Save", "Download", "Share", "Schedule", "Open in", and "More". The query is written in a dark-themed editor. It consists of two main parts: a CTE named "VIP_Stats" and a final SELECT statement. The CTE selects user_id, total_gastat (sum of amount), num_compres (count of transactions), tiquet_mig (average amount rounded to 2), and max_compra (maximum amount) from a table named "sprint3-analytics-jordi-m.sprint3_silver.transactions_clean", filtered by "declined = 0" and grouped by "user_id", with a HAVING clause for "SUM(amount) > 500". The final SELECT statement joins the CTE with a table named "sprint3-analytics-jordi-m.sprint3_silver.users_combined" on "user_id" and selects user details (user_id, name, surname, email) along with the aggregated statistics from the CTE, ordered by "vip.total_gastat" in descending order. A status bar at the bottom indicates "Query completed" and "Using on-demand processing quota".

```
1 WITH VIP_Stats AS (  
2     SELECT  
3         user_id,  
4         SUM(amount) AS total_gastat,  
5         COUNT(*) AS num_compres,  
6         ROUND(AVG(amount), 2) AS tiquet_mig,  
7         MAX(amount) AS max_compra  
8     FROM `sprint3-analytics-jordi-m.sprint3_silver.transactions_clean`  
9     WHERE declined = 0  
10    GROUP BY user_id  
11    HAVING SUM(amount) > 500  
12 )  
13  
14 SELECT  
15     u.user_id,  
16     CONCAT(u.name, ' ', u.surname) AS nom_complet,  
17     u.email,  
18     vip.num_compres,  
19     vip.tiquet_mig,  
20     vip.max_compra,  
21     vip.total_gastat  
22 FROM VIP_Stats AS vip  
23 JOIN `sprint3-analytics-jordi-m.sprint3_silver.users_combined` AS u  
24 ON vip.user_id = u.user_id  
25 ORDER BY vip.total_gastat DESC;
```

Creamos una vista para tener localizados a los clientes VIP, que son aquellos clientes que han gastado más de 500€. Tener localizados éstos clientes VIP puede ser muy beneficioso, tanto para crear campañas de marketing dirigidas a éstos clientes, como analizar maneras e incentivar a los clientes que ahora mismo no son VIP a que alcancen dicho estatus.

Untitled query		Run	Save	Download	Share	Schedule	Open in	More
<pre> 1 WITH VIP_Stats AS (2 SELECT 3 user_id, 4 SUM(amount) AS total_nastat </pre>								
Query completed								
Using on-demand processing quota								
Query results		Chat	View all jobs	Save results	Open in			
Job information	Results	Visualisation	JSON	Execution details	Execution graph			
user_id	nom_complet	email	num_compres	tiquet_mig	max_compra	total_gastat		
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16	
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449	615.51	38614.39	
3	455	Fwfl Phylgmhnd	fwfl.phylgmhnd@example.com	66	522.6	724.44	34491.27	
4	417	Fqatil Rdwcjvoa	fqatil.rdwcjvoa@example.com	73	421.87	675.78	30796.3	
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.79	
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51	
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646	25485.05	
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	78	304	604.9	23711.9	
9	349	Yxcjk Fnceqbkrmt	yxcjk.fnceqbkrmt@example.c...	53	443.48	675.05	23504.47	
10	317	Qikcvc Nvfcifqdh	qikcvc.nvfcifqdh@example.com	54	351.69	600.77	18991.45	
11	388	Rbjupnac Cfjykd	rbjupnac.cfjykd@example.com	62	297.3	479.03	18432.84	
12	319	Fxtg Eedgl	fxtg.eedgl@example.com	62	286.62	576.22	17770.39	
13	443	Qimzjbfq Cvpdvtz	qimzjbfq.cvpdvtz@example.com	75	230.81	524.33	17310.42	
14	393	Nxbq Bmbzxiihu	nxbq.bmbzxiihu@example.com	76	220.09	534.22	16726.47	
Results per page: 50 1 - 50 of 5000								

Vemos que actualmente tenemos 5000 clientes VIP, y podemos ver a simple vista su información personal, cuántas compras han realizado, el promedio por compra, la compra más costosa que han realizado, y el total gastado. La consulta no está ordenada.

Caso 2. Análisis de tendencias de ventas

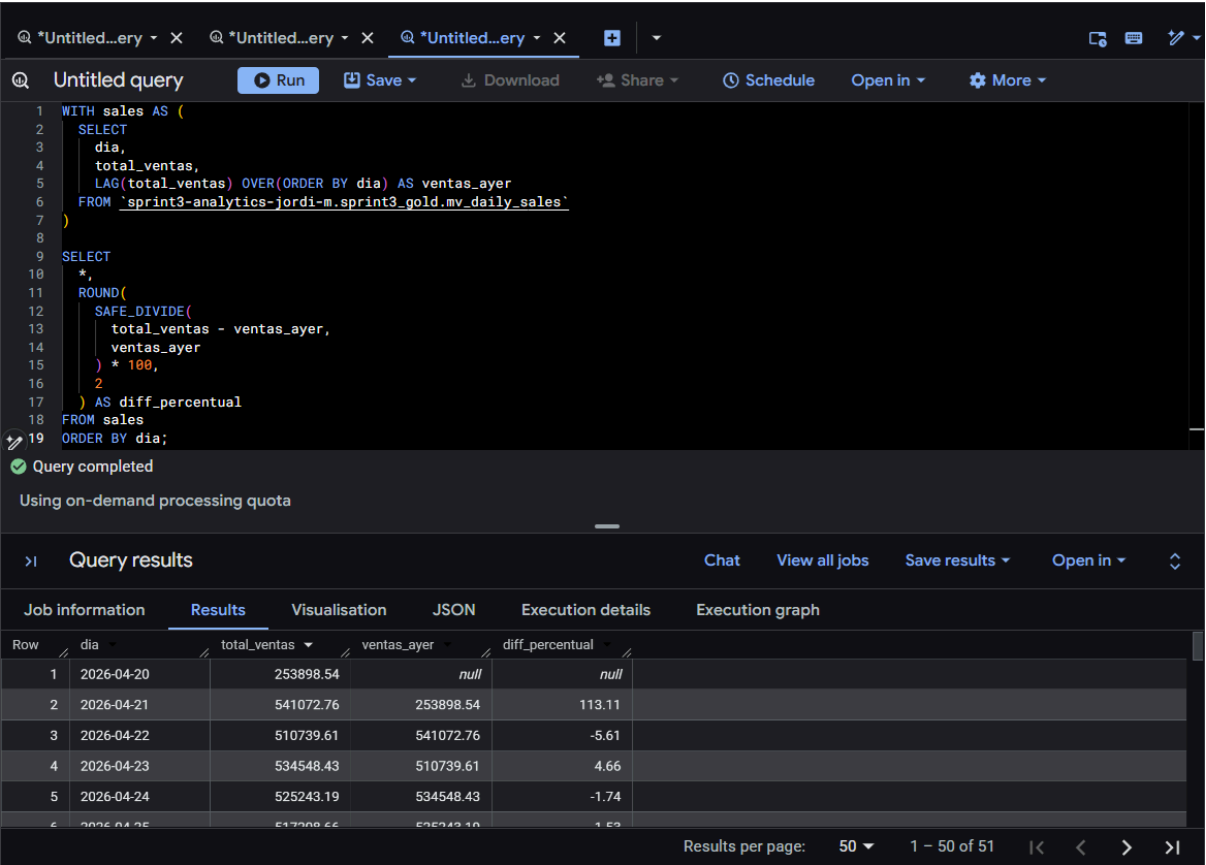
Contexto

La dirección financiera necesita monitorizar la evolución diaria del negocio.

Objetivo

Comparar cada día con el anterior para detectar:

- Crecimientos.
- Caídas.
- Comportamientos anómalos.



The screenshot shows a SQL query editor interface with a query that calculates the percentage change in daily sales. The query uses a window function (LAG) to compare the current day's sales with the previous day's sales. Below the query, a message indicates the query completed successfully. At the bottom, a table displays the results of the query, showing columns for the date (dia), total sales (total_ventas), sales from the previous day (ventas_ayer), and the percentage change (diff_percentual).

```
1 WITH sales AS (  
2   SELECT  
3     dia,  
4     total_ventas,  
5     LAG(total_ventas) OVER(ORDER BY dia) AS ventas_ayer  
6   FROM `sprint3-analytics-jordi-m.sprint3_gold_mv_daily_sales`  
7 )  
8  
9 SELECT  
10  *,  
11  ROUND(  
12    SAFE_DIVIDE(  
13      total_ventas - ventas_ayer,  
14      ventas_ayer  
15    ) * 100,  
16    2  
17  ) AS diff_percentual  
18 FROM sales  
19 ORDER BY dia;
```

Query completed
Using on-demand processing quota

Query results

Job information	Results	Visualisation	JSON	Execution details	Execution graph
Row	dia	total_ventas	ventas_ayer	diff_percentual	
1	2026-04-20	253898.54	null	null	
2	2026-04-21	541072.76	253898.54	113.11	
3	2026-04-22	510739.61	541072.76	-5.61	
4	2026-04-23	534548.43	510739.61	4.66	
5	2026-04-24	525243.19	534548.43	-1.74	
6	2026-04-25	517308.66	525243.19	-1.53	

Results per page: 50 1 - 50 of 51

Utilizando la vista materializada que hicimos anteriormente, creamos una consulta para monitorizar la velocidad de las ventas, es decir, que compare el rendimiento de cada día contra el día anterior para detectar caídas bruscas o picos de crecimiento. Con esta información, podríamos utilizar filtros estableciendo valores por los que filtrar caídas o picos superiores a un 10%, por ejemplo. Con dichos filtros podemos extraer información relevante e incluso identificar patrones.

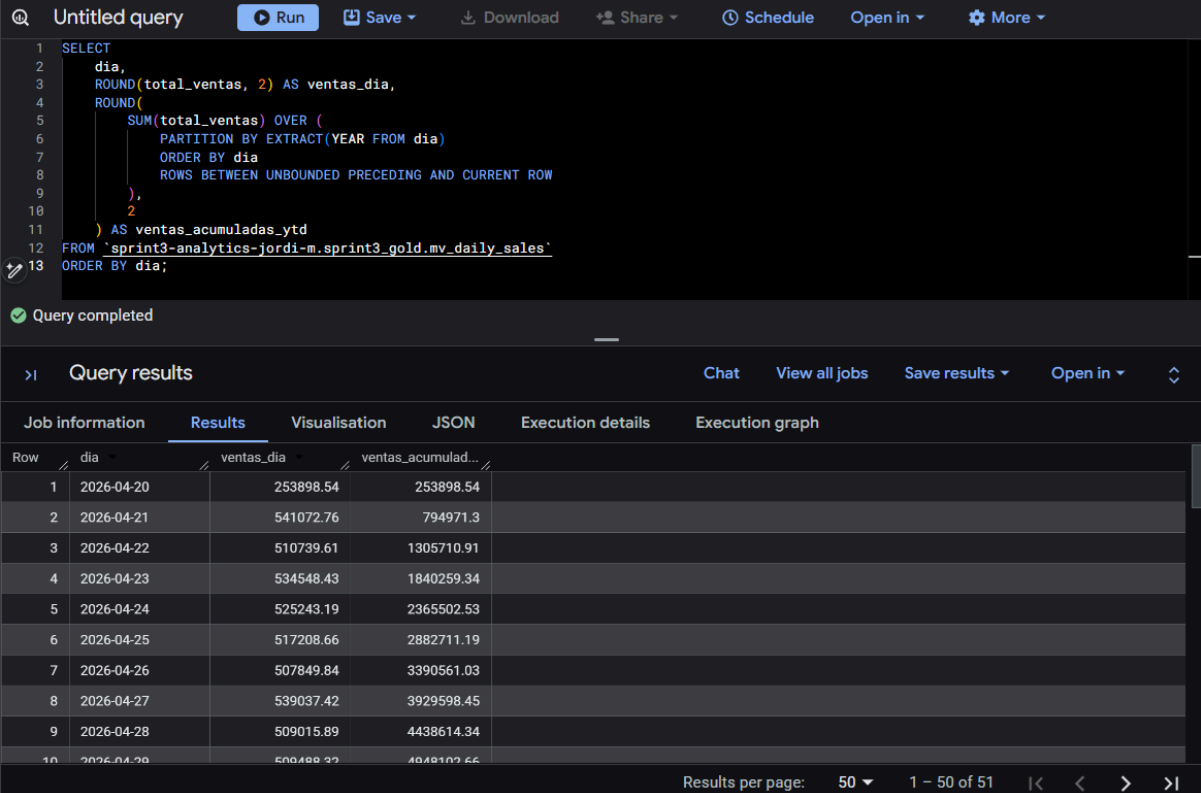
Caso 3. Evolución acumulada de ingresos

Contexto

El CFO necesita visualizar el crecimiento acumulado del negocio a lo largo del año.

Objetivo

Calcular ventas acumuladas YTD (Year-To-Date).



The screenshot shows a SQL query in a dark-themed editor. The query selects the date (dia), daily sales (ventas_dia), and cumulative sales (ventas_acumuladas_ytd) from a table named 'sprint3-analytics-jordi-m.sprint3_gold.mv_daily_sales'. The cumulative sales are calculated using a window function: SUM(total_ventas) OVER (PARTITION BY EXTRACT(YEAR FROM dia) ORDER BY dia ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW). The results are displayed in a table with 51 rows. The first 10 rows are visible, showing dates from 2026-04-20 to 2026-04-29. The table has columns for Row, dia, ventas_dia, and ventas_acumuladas_ytd. The results show a steady increase in daily sales, with the cumulative total reaching 4948102.66 by the end of the period shown.

```
1 SELECT
2   dia,
3   ROUND(total_ventas, 2) AS ventas_dia,
4   ROUND(
5     SUM(total_ventas) OVER (
6       PARTITION BY EXTRACT(YEAR FROM dia)
7       ORDER BY dia
8       ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
9     ), 2
10  ) AS ventas_acumuladas_ytd
11 FROM 'sprint3-analytics-jordi-m.sprint3_gold.mv_daily_sales'
12 ORDER BY dia;
```

Query completed

Query results

Job information Results Visualisation JSON Execution details Execution graph

Row	dia	ventas_dia	ventas_acumuladas_ytd
1	2026-04-20	253898.54	253898.54
2	2026-04-21	541072.76	794971.3
3	2026-04-22	510739.61	1305710.91
4	2026-04-23	534548.43	1840259.34
5	2026-04-24	525243.19	2365502.53
6	2026-04-25	517208.66	2882711.19
7	2026-04-26	507849.84	3390561.03
8	2026-04-27	539037.42	3929598.45
9	2026-04-28	509015.89	4438614.34
10	2026-04-29	500488.32	4948102.66

Results per page: 50 1 - 50 of 51

Utilizando la vista materializada, hacemos una consulta para ver cual es la curva de crecimiento anual.

Caso 4. Análisis de fidelización y tercera compra

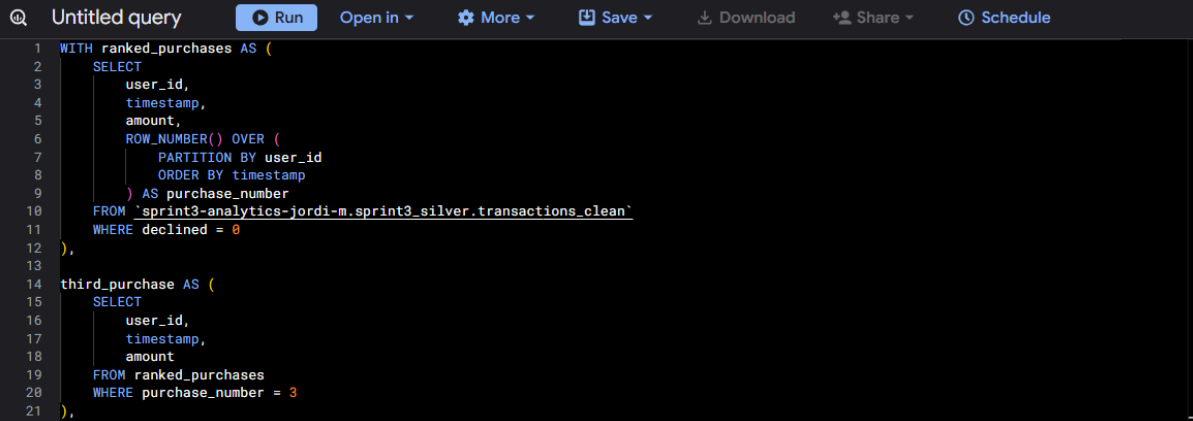
Contexto

Marketing quiere activar una campaña específica cuando un usuario alcanza su tercera compra.

Objetivo

Identificar:

- Fecha de la tercera compra.
- Importe de la tercera compra.
- Ticket medio de las tres primeras compras.



```
1 WITH ranked_purchases AS (  
2     SELECT  
3         user_id,  
4         timestamp,  
5         amount,  
6         ROW_NUMBER() OVER (  
7             PARTITION BY user_id  
8             ORDER BY timestamp  
9         ) AS purchase_number  
10    FROM `sprint3-analytics-jordi-m.sprint3_silver.transactions_clean`  
11   WHERE declined = 0  
12 ),  
13  
14 third_purchase AS (  
15     SELECT  
16         user_id,  
17         timestamp,  
18         amount  
19     FROM ranked_purchases  
20    WHERE purchase_number = 3  
21 ),
```

Query completed
Using on-demand processing quota

Query results

Job information	Results	Visualisation	JSON	Execution details	Execution graph	
Row	user_id	nom_complet	email	date_third_purchase	amount_third_pur...	avg_first_three
1	1867	Wplbds Ejuchosc	wplbds.ejuchosc@example.com	2015-09-28 05:04:57 UTC	263.26	210.91
2	4278	Lwcdof Isruystb	lwcdof.isruystb@example.com	2017-11-24 19:11:45 UTC	107.17	117.63
3	1497	Etcuyh Irysyzge	etcuyh.iryzyge@example.com	2016-09-16 08:31:35 UTC	169.96	251.61
4	3739	Dfcwtz Gnsombfd	dfcwtz.ansombfd@example.com	2016-12-26 19:34:43 UTC	167.52	125.88

Results per page: 50 1 - 50 of 5000

Primera mitad del código para la consulta

Untitled query Run Open in More Save Download Share Schedule

```

23 first_three_avg AS (
24     SELECT
25         user_id,
26         ROUND(AVG(amount), 2) AS avg_first_three
27     FROM ranked_purchases
28     WHERE purchase_number <= 3
29     GROUP BY user_id
30 )
31
32 SELECT
33     u.user_id,
34     CONCAT(u.name, ' ', u.surname) AS nom_complet,
35     u.email,
36     tp.timestamp AS date_third_purchase,
37     tp.amount AS amount_third_purchase,
38     f.avg_first_three
39 FROM `sprint3-analytics-jordi-m.sprint3_silver.users_combined` AS u
40 JOIN third_purchase AS tp
41     ON u.user_id = tp.user_id
42 JOIN first_three_avg AS f
43     ON u.user_id = f.user_id;

```

Query completed

Using on-demand processing quota

Query results Chat View all jobs Save results Open in

Job information **Results** Visualisation JSON Execution details Execution graph

Row	user_id	nom_complet	email	date_third_purchase	amount_third_pur...	avg_first_three
1	1867	Wplbds Ejuchosc	wplbds.ejuchosc@example.com	2015-09-28 05:04:57 UTC	263.26	210.91
2	4278	Lwcdof Isruystb	lwcdof.isruystb@example.com	2017-11-24 19:11:45 UTC	107.17	117.63
3	1497	Etcuyh Irysyge	etcuyh.iryzyge@example.com	2016-09-16 08:31:35 UTC	169.96	251.61
4	3739	Dfcwtz Gnsombfd	dfcwtz.ansombfd@example.com	2016-12-26 19:34:43 UTC	167.52	125.88

Results per page: 50 1 - 50 of 5000

Segunda mitad del código para la consulta. Creamos tres tablas intermedias, una para tener la información de las ventas con el orden de las ventas “agrupadas” por usuario. De ahí, para saber cuánto gastaron en la tercera compra aplicamos un filtro donde purchase_number sea igual a 3. Para saber la media de las tres primeras compras, aplicamos un filtro donde purchase_number sea igual o inferior a 3, y calcularemos la media de importe agrupando por usuario. Finalmente juntamos toda la información en una tabla. Este proceso es más eficiente que utilizar QUALIFY, ya que para utilizarlo tendríamos que utilizar ROW_NUMBER() dos veces, y por lo tanto crear dos consultas.

Untitled queryRunOpen inMoreSaveDownloadShareSchedule

Query completed

Using on-demand processing quota

>|Query resultsChatView all jobsSave resultsOpen in

Job informationResultsVisualisationJSONExecution detailsExecution graph

Row	user_id	nom_complet	email	date_third_purchase	amount_third_pur...	avg_first_three
1	1867	Wplbds Ejuchosc	wplbds.ejuchosc@example.com	2015-09-28 05:04:57 UTC	263.26	210.91
2	4278	Lwcdof Isruystb	lwcdof.isruystb@example.com	2017-11-24 19:11:45 UTC	107.17	117.63
3	1497	Etcuyh Irysydze	etcuyh.iryzydze@example.com	2016-09-16 08:31:35 UTC	169.96	251.61
4	3739	Dfcwtz Gnsombfd	dfcwtz.gnsombfd@example.com	2016-12-26 19:34:43 UTC	167.52	125.88
5	4769	Qmnyyc Otihibkl	qmnyyc.otihzbkl@example.com	2018-06-06 15:33:03 UTC	424.58	388.96
6	3932	Amhiax Irmqozry	amhiax.irmqozry@example.com	2016-11-05 04:06:17 UTC	151.78	178.54
7	1401	Devnbq Zlqgsuu	devnbq.zlqgsuu@example.com	2015-08-02 00:18:50 UTC	427.1	284.17
8	2307	Dviexc Phcaqseq	dviexc.phcaqseq@example.com	2016-09-15 00:59:40 UTC	201.16	180.73
9	4665	Trjwor Mnjtkrgj	trjwor.mnjtkrgj@example.com	2016-10-12 00:27:38 UTC	26.66	94.89
10	2914	Uwyxuu Exdinpfc	uwyxuu.exdinpfc@example.com	2015-07-19 09:49:13 UTC	257.27	306.81
11	3553	Evblkj Prkykddd	evblkj.prkykddd@example.com	2016-12-06 01:31:25 UTC	12.45	117.8
12	821	Sckoxi Xthxvtak	sckoxi.xthxvtak@example.com	2016-01-19 12:31:45 UTC	372.66	219.77
13	4233	Ofcqi Yochyebis	ofcqi.yochyebis@example.com	2017-12-11 15:03:08 UTC	328.44	233.18
14	907	Zwtyhn Solssgik	zwtyhn.solssgik@example.com	2017-01-20 12:43:51 UTC	96.9	224.6
15	2013	Erswn Ohijzach	erswn.ohijzach@example.com	2016-03-02 03:34:42 UTC	127.09	180.88
16	231	Zoe Morrow	lacinia@google.com	2016-04-11 09:10:29 UTC	339.63	234.17
17	905	Dimeu Iustbud	dimeu.iustbud@example.com	2015-02-08 12:02:37 UTC	26.51	100.97

Results per page: 501 - 50 of 5000|<>>|

Visualización de los resultados de la consulta. Esta consulta nos devuelve 5000 registros.

Nivel 3 - Analytics Engineering

Caso 1. Desnormalización de transacciones para análisis de producto

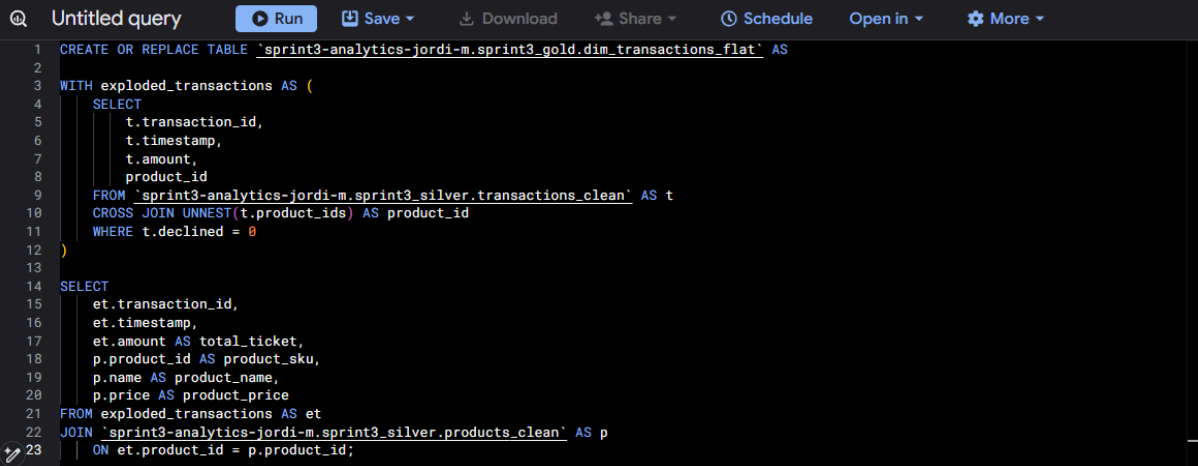
Contexto

Los productos vendidos se almacenan en arrays dentro de cada transacción.

Este formato dificulta el análisis comercial.

Objetivo

Transformar los arrays en una tabla plana preparada para análisis.



```
1 CREATE OR REPLACE TABLE `sprint3-analytics-jordi-m.sprint3_gold.dim_transactions_flat` AS
2
3 WITH exploded_transactions AS (
4     SELECT
5         t.transaction_id,
6         t.timestamp,
7         t.amount,
8         product_id
9     FROM `sprint3-analytics-jordi-m.sprint3_silver.transactions_clean` AS t
10    CROSS JOIN UNNEST(t.product_ids) AS product_id
11    WHERE t.declined = 0
12 )
13
14 SELECT
15     et.transaction_id,
16     et.timestamp,
17     et.amount AS total_ticket,
18     p.product_id AS product_sku,
19     p.name AS product_name,
20     p.price AS product_price
21 FROM exploded_transactions AS et
22 JOIN `sprint3-analytics-jordi-m.sprint3_silver.products_clean` AS p
23   ON et.product_id = p.product_id;
```

Query completed
Using on-demand processing quota

Query results Chat View all jobs Save results Open in

Job information Results Execution details Execution graph

This statement created a new table named dim_transactions_flat. Go to table

Creamos una nueva tabla donde desanidaremos la información que previamente guardamos en `product_ids` en forma de array. Para ello utilizaremos un `CROSS JOIN UNNEST`. Sólo lo aplicaremos a aquellas transacciones que no hayan sido rechazadas.

Esto nos permitirá desglosar cada transacción de manera que la información se repita en función de cuántos productos se compraron en cada transacción. Esto nos permite ver más claramente cuál es el rendimiento de cada producto, así como identificar otros patrones con respecto a la información desglosada, como por ejemplo patrones de compra de productos por origen (si cruzamos la información con la de `users_combined` y origen).

Schema

Field name	Type	Mode	Description	Key	Collation	Default value	Policy tags	Data policies
transaction_id	STRING	NULLABLE	-	-	-	-	-	-
timestamp	TIMESTAMP	NULLABLE	-	-	-	-	-	-
total_ticket	NUMERIC	NULLABLE	-	-	-	-	-	-
product_sku	INTEGER	NULLABLE	-	-	-	-	-	-
product_name	STRING	NULLABLE	-	-	-	-	-	-
product_price	NUMERIC	NULLABLE	-	-	-	-	-	-

Edit schema View row access policies Describe data

Vemos cómo ha quedado el esquema de la tabla después de la transformación.

Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualisation	JSON	Execution details	Execution graph	
Row	transaction_id	timestamp	total_ticket	product_id	product_name	product_price
1	00043A49-2949-494B-A5DD-A5...	2024-08-28 07:16:46 UTC	395.43	87	sith Jade	96.26
2	00043A49-2949-494B-A5DD-A5...	2024-08-28 07:16:46 UTC	395.43	16	the duel warden	180.91
3	00043A49-2949-494B-A5DD-A5...	2024-08-28 07:16:46 UTC	395.43	97	jinn Winterfell	65.25
4	00043A49-2949-494B-A5DD-A5...	2024-08-28 07:16:46 UTC	395.43	26	Stark Karstark	53.01
5	000447FE-B650-4DCF-85DE-C7...	2016-12-21 20:07:18 UTC	155.63	87	sith Jade	96.26
6	000447FE-B650-4DCF-85DE-C7...	2016-12-21 20:07:18 UTC	155.63	66	mustafar jinn	2.12
7	000447FE-B650-4DCF-85DE-C7...	2016-12-21 20:07:18 UTC	155.63	69	Littlefinger the giantsblood	57.25
8	00045D6B-ED2E-4F2F-8186-CE...	2020-07-14 15:37:45 UTC	326.01	16	the duel warden	180.91
9	00045D6B-ED2E-4F2F-8186-CE...	2020-07-14 15:37:45 UTC	326.01	30	Karstark warden	79.53
10	00045D6B-ED2E-4F2F-8186-CE...	2020-07-14 15:37:45 UTC	326.01	11	Karstark Dorne	49.7
11	00045D6B-ED2E-4F2F-8186-CE...	2020-07-14 15:37:45 UTC	326.01	81	the duel	15.87
12	000481C3-1C26-4FEF-83A0-4C...	2017-09-04 19:44:53 UTC	161.6	72	north	161.6
13	00051AA4-9CBE-4268-B070-C3...	2017-01-05 18:19:25 UTC	148.91	18	Karstark warden	148.91
14	0008A312-EDFE-4A4F-BC99-E9...	2023-09-23 04:51:43 UTC	294.59	33	duel warden	127.09

Results per page: 50 1 - 50 of 252693

Una vez desanidadas las transacciones, tenemos un total de 252693 registros de transacciones que no han sido rechazadas.

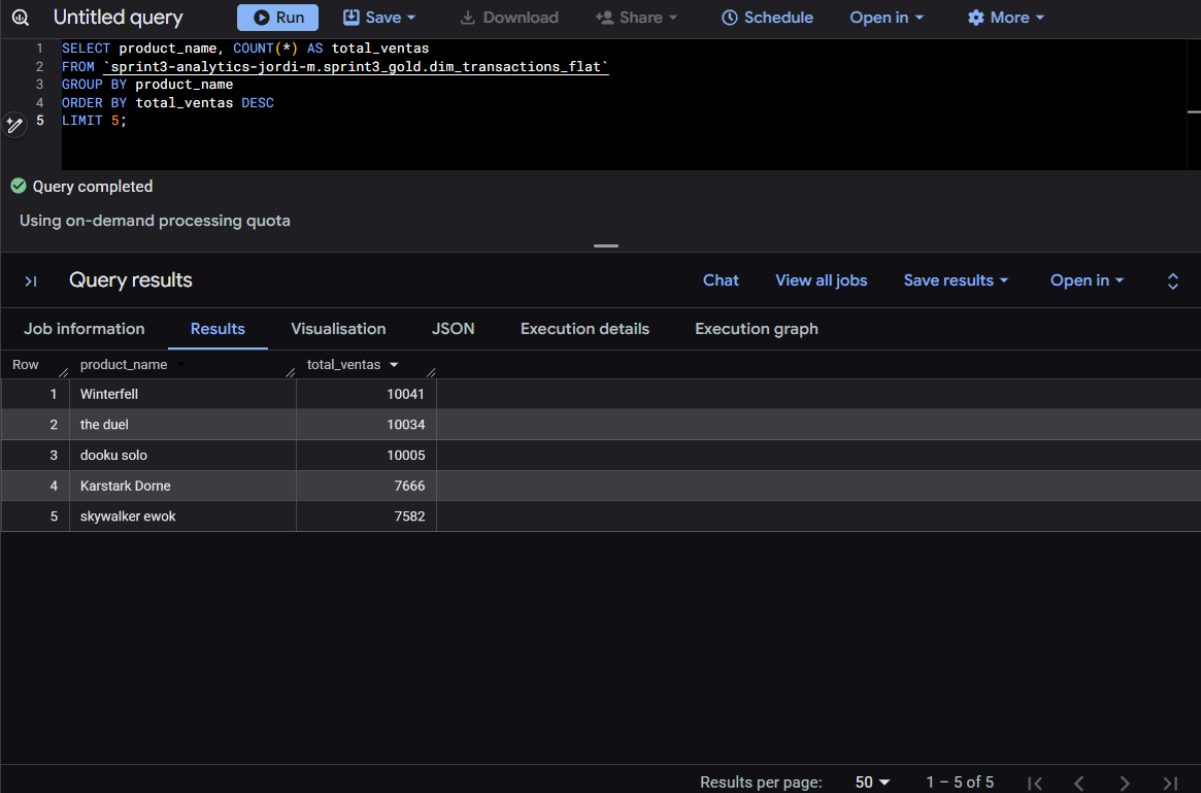
Caso 2. Ranking histórico de productos

Contexto

El equipo de ventas necesita identificar los productos más demandados.

Objetivo

Generar un ranking de productos basado en unidades vendidas.



The screenshot shows a SQL query editor with the following query:

```
1 SELECT product_name, COUNT(*) AS total_ventas
2 FROM 'sprint3-analytics-jordi-m.sprint3_gold.dim_transactions_flat'
3 GROUP BY product_name
4 ORDER BY total_ventas DESC
5 LIMIT 5;
```

Below the query editor, a message indicates "Query completed" and "Using on-demand processing quota".

The "Query results" section shows a table with the following data:

Row	product_name	total_ventas
1	Winterfell	10041
2	the duel	10034
3	dooku solo	10005
4	Karstark Dome	7666
5	skywalker ewok	7582

At the bottom, the interface shows "Results per page: 50" and "1 - 5 of 5".

Utilizando la tabla creada anteriormente, podemos generar un ranking de los productos más vendidos y ver cuánto se ha vendido cada producto en total. Para esta consulta nos quedamos con los cinco productos más vendidos.

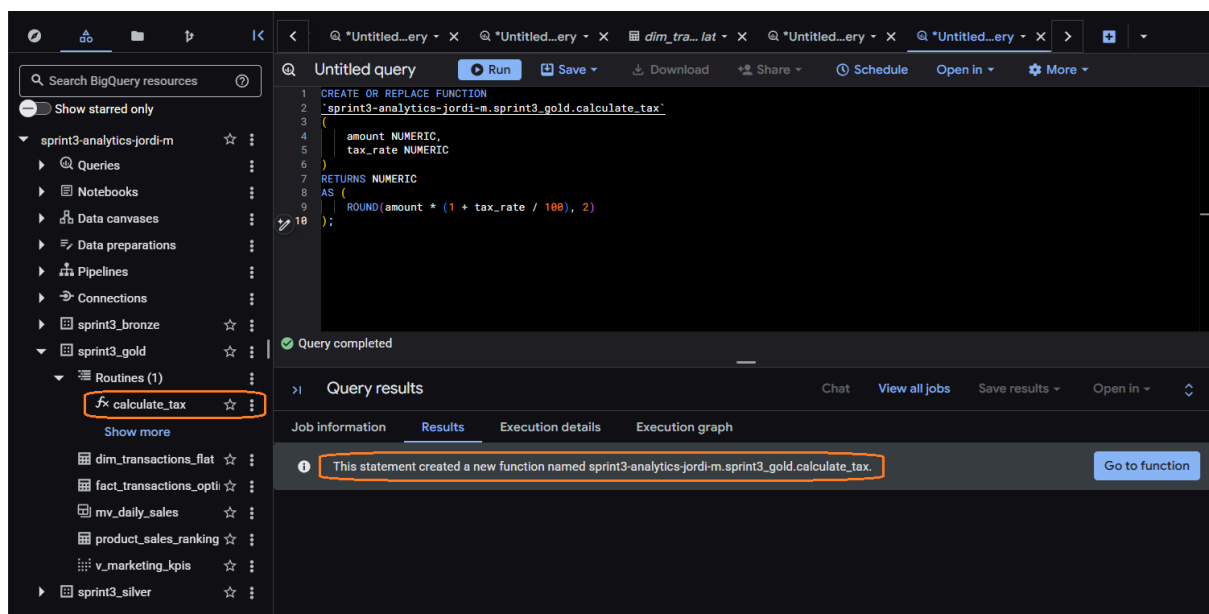
Caso 3. Automatización del pipeline analítico

Contexto

Marketing necesita disponer diariamente de la información actualizada sin intervención manual.

Objetivos

- Centralizar la lógica fiscal mediante una UDF.
- Automatizar la regeneración de tablas.
- Construir un dashboard en Data Studio (antes Looker Studio).



Creamos una función que tome dos parámetros de entrada, que serán el amount, y otro que será el valor del IVA. Utilizamos valores NUMERIC, ya que para valores monetarios generan menos errores. La función nos devolverá el valor de la transacción con IVA incluido, redondeado a 2 decimales.

The screenshot shows a SQL query editor with the following SQL code:

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-jordi-m.sprint3_gold.dim_transactions_flat` AS
2
3 WITH exploded_transactions AS (
4   SELECT
5     t.transaction_id,
6     t.timestamp,
7     t.amount,
8     product_id
9   FROM `sprint3-analytics-jordi-m.sprint3_silver.transactions_clean` AS t
10  CROSS JOIN UNNEST(t.product_ids) AS product_id
11  WHERE t.declined = 0
12 )
13
14 SELECT
15   et.transaction_id,
16   et.timestamp,
17   et.amount AS total_ticket,
18   p.product_id AS product_sku,
19   p.name AS product_name,
20   p.price AS product_price,
21   `sprint3-analytics-jordi-m.sprint3_gold.calculate_tax` (
22     p.price,
23     21
24   ) AS product_price_tax_inc
25 FROM exploded_transactions AS et
26 JOIN `sprint3-analytics-jordi-m.sprint3_silver.products_clean` AS p
27   ON et.product_id = p.product_id;
```

Below the query, a status bar indicates "Query completed" and "Using on-demand processing quota". At the bottom, there is a "Query results" section with tabs for "Job information", "Results", "Execution details", and "Execution graph". A message states: "This statement replaced the table named dim_transactions_flat." with a "Go to table" button.

Reemplazamos la tabla, esta vez sacando el valor de la transacción con el IVA incluido, y encuadrado dentro de la función, para este ejercicio, le damos un valor del 21%.

Para agendar la que la vista se actualice cada día a una hora determinada, pinchamos el botón de 'Schedule', marcado en naranja en la parte superior derecha de la imagen.

Search (/) for resources, docs, products and more

Q *Untitled...ery X dim_tra... lat X Q *Untitled...ery X

Q Untitled query Run Save Download +

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-jordi-m.sprint3_gold.dim_transactions_flat`
2
3 WITH exploded_transactions AS (
4   SELECT
5     t.transaction_id,
6     t.timestamp,
7     t.amount,
8     product_id
9   FROM `sprint3-analytics-jordi-m.sprint3_silver.transactions_clean` AS t
10  CROSS JOIN UNNEST(t.product_ids) AS product_id
11  WHERE t.declined = 0
12 )
13
14 SELECT
15   et.transaction_id,
16   et.timestamp,
17   et.amount AS total_ticket,
18   p.product_id AS product_sku,
19   p.name AS product_name,
20   p.price AS product_price,
21   `sprint3-analytics-jordi-m.sprint3_gold.calculate_tax` (
22     p.price,
23     21
24   ) AS product_price_tax_inc
25 FROM exploded_transactions AS et
26 JOIN `sprint3-analytics-jordi-m.sprint3_silver.products_clean` AS p
27   ON et.product_id = p.product_id;
```

Query completed

Using on-demand processing quota

Query results

Job information Results Execution details Execution graph

This statement replaced the table named dim_transactions_flat.

Export as scheduled query

Settings valid.

Details and schedule

Name for scheduled query *
refresh_dim_transactions_flat

Schedule options

Repeat frequency *
Days

At *
07:00 UTC

☒ Start now ☐ Start at set time

Start date and run time
09/06/2026, 19:22 CET

☒ End never ☐ Schedule end time

End date
CET

Scheduled to run every day at 07:00 UTC starting now.

Destination for query results

☐ Set a destination table for query results

Save Cancel

Se nos desplegará desde la derecha un menú. Rellenamos con el nombre del proceso, la frecuencia con la que queremos que se actualicen los datos, y en el caso de que sea diariamente, la hora (en formato UTC) a la que queremos que se actualice. Para este caso, no tocaremos más opciones. Finalmente guardaremos el proceso.

Para terminar, adjuntamos un enlace para visualizar a través de Data Studio (antes Looker Studio) métricas clave de los datos que hemos trabajado en los últimos Sprints con BigQuery:

<https://datastudio.google.com/s/qXOutl-bvkc>

Informe sin título

RestablecerCompartirVer

Añadir páginaAñadir datosCombinarAñadir un gráficoAñadir un controlTema y diseñoPausar actualizaciones

Añadir datos al informe

Credenciales de acceso a datos: J.J. Montseny

Haga que sus informes de BigQuery se carguen más rápido con BigQuery BI Engine. Más información

BigQuery

De Google

BigQuery es un almacén de datos de análisis de bajo coste administrado íntegramente por Google y en el que se utiliza una escala de petabytes. Este servicio se cobra por consulta o procesamiento de datos, y el importe de esas consultas se carga en la tarjeta de crédito del proyecto de facturación.

MÁS INFORMACIÓNNOTIFICAR UN PROBLEMA

PROYECTOS RECIENTES

MIS PROYECTOS

PROYECTOS COMPARTIDOS

CONSULTA PERSONALIZADA

CONJUNTOS DE DATOS PÚBLICOS

Project

Search Projects

Enter Project Id manually

sprint3-analytics-jordi-m

Conjunto de datos

Search Datasets

sprint3_bronze

sprint3_gold

sprint3_silver

Table

Search Tables

dm_transactions_flat

fact_transactions_optimized

mv_daily_sales

product_sales_ranking

v_marketing_kpis

Cancelar

Añadir

Ingresos Totales (IVA Incluido)

31.254.761,80 €

Transacciones Únicas

99.763

Ticket Medio

313,29 €

product_price_tax_inc

20 mil

15 mil

10 mil

5 mil

0

1 ene 20151 sept 201530 dic 201630 abr 201829 dic 201829 ago 201928 abr 202027 dic 202027 ago 202126 dic 202226 ago 202325 abr 202424 dic 2024

product_nameUnidades Vendidas

skywalker ewok7.582

the duel10.034

Karstark warden7.539

Stannis warden5.005

north5.127

Winterfell10.041

1 - 70 / 70

product_nameIngresos Totales

skywalker ewok1.533.910,33 €

the duel1.270.666,43 €

Karstark warden1.243.414,99 €

Stannis warden1.099.168,63 €

north1.054.174,20 €

Winterfell1.052.045,15 €

1 - 70 / 70

product_nameIVA Recaudado

skywalker ewok266.213,20 €

the duel220.521,82 €

Karstark warden215.797,48 €

Stannis warden190.756,98 €

north182.966,88 €

Winterfell182.581,45 €

1 - 70 / 70